

Homework 2

Being Familiar with Numpy, Pandas, and Matplotlib

Project Overview

Create a data analysis application that **simulates analyzing university course performance data**. In this homework, you will generate synthetic data, clean and process it using Pandas, perform calculations with NumPy, and create various visualizations with Matplotlib.

Learning Objectives

This homework provides a comprehensive assessment of the key concepts covered in the course while creating a practical, real-world data analysis scenario. It evaluates students' understanding of:

- **NumPy**: Array creation with random values, statistical functions, array masking, reshaping
- **Pandas**: DataFrame creation, handling missing values, groupby operations, merging, aggregation functions
- **Matplotlib**: Multiple plot types (bar, histogram, scatter, pie, box, heatmap, line), subplots, styling

Expected Learning Outcomes

Upon completing this project, students will demonstrate proficiency in:

- Generating and manipulating synthetic datasets
- Cleaning and preprocessing data
- Performing statistical analysis
- Creating informative visualizations
- Interpreting results through multiple visualization types

Project Requirements

1. **Data Generation** (NumPy & Pandas)

- Create student records with random data
- Generate course enrollment information
- Simulate exam scores and assignment grades

2. **Data Processing** (Pandas)

- Handle missing values
- Merge datasets
- Calculate derived metrics

3. Analysis & Visualization (Matplotlib)

- Create multiple plot types to showcase different aspects of the data

Detailed Specifications

Part 1: Data Generation with Numpy

1. Create two arrays of random normally distributed values representing scores of two exams (midterm and final) for 100 students. Assume that all the scores are between 0 and 100.
2. Create an array of 500 random normally distributed values representing assignment scores (5 assignments for 100 students, each between 0 and 100). Then, reshape this array to have 100 rows and 5 columns.
3. Create an array of random choices for a categorical variable (e.g., 'Department') from ['CS', 'EE', 'PHYS', 'CHEM', 'BIO'] for each student.
4. Introduce some missing values (NaN) in randomly created arrays (i.e. midterm, final, assignments, and departments). After this step, 5 percent of the data should be NaNs. For example, you can randomly specify some indices and replace the values of these indices with NaNs. A sample code for this step is as follows:

```
In [ ]: missing_indices = np.random.choice(100, size=5)
scores[missing_indices] = np.nan
```

Part 2: DataFrames with Pandas

1. Create a DataFrame (df1) with columns: 'StudentID', 'Department', 'Midterm', 'Final'.
2. Add the 5 columns for assignments to the above dataframe.

After these steps, you will have a DataFrame like this (Only the first 5 rows among 100 are listed here):

```
Initial Student Data:
  StudentID Department  Midterm  Final  Assignment_1  Assignment_2 \
0      1000      PHYS    94.1    55.1           76.4           85.1
1      1001      PHYS    64.8    72.9           79.5           88.7
2      1002      nan    46.5    95.7           73.5           55.2
3      1003      EE      NaN    84.3           75.4           86.0
4      1004      CS     66.8    64.8           82.1           90.0

  Assignment_3  Assignment_4  Assignment_5
0          88.7           61.5           65.9
1          84.1           77.1           89.5
2          92.5           57.6           88.2
3          67.7           51.9           87.1
4          74.9           68.6           73.6

Dataset shape: (100, 9)
```

Dataset shape: (100, 9)

3. Handle the missing values:

- For the categorical column (Department), fill with the mode (most frequent department).

- For numeric columns (Midterm, Final, Assignments), fill with the mean of the column for the same department.

Part 3: Data Aggregation and Grouping

1. Compute the total score for each student: Midterm (25%), Final (25%), and the average of assignments (50%).
2. Add a column 'Grade' which is a letter grade based on the total score:
A: 90-100, B: 80-89, C: 70-79, D: 60-69, F: <60.
3. Group by 'Department' and compute the average and standard deviation for the total score, the average of midterm and final exams, and the count of students in each department.

At the end of this part, you should have a DataFrame like this:

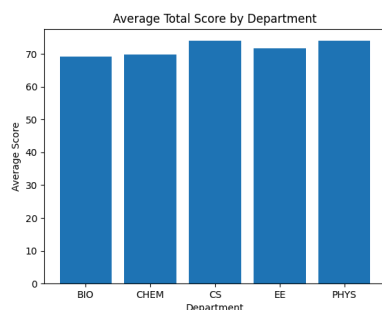
Department Summary:

	Department	Avg_Total	Std_Total	Count	Avg_Midterm	Avg_Final
0	BIO	75.72	7.09	13	83.05	68.19
1	CHEM	70.74	6.83	29	71.70	62.58
2	CS	70.38	6.14	20	72.49	62.18
3	EE	70.16	7.44	20	66.12	68.12
4	PHYS	71.82	7.05	18	75.76	65.36

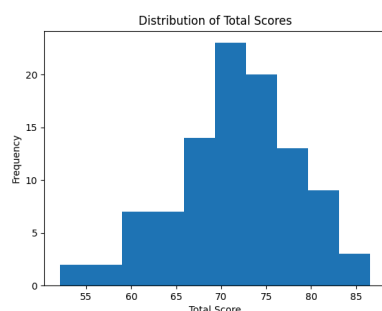
Part 4: Visualization with Matplotlib

Draw the following charts and plots:

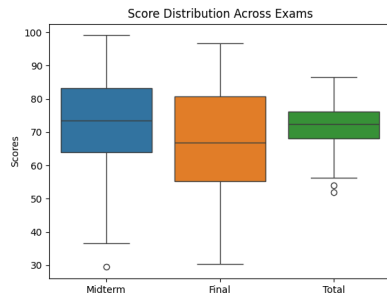
1. Bar chart showing the average total score for each department. Sample:



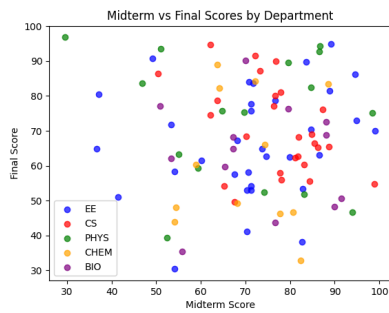
2. Histogram of the total scores. Sample:



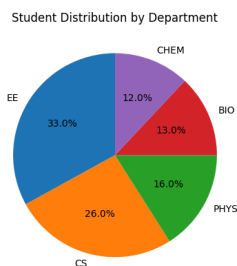
3. Boxplot of the total, midterm, and final scores by department. Sample:



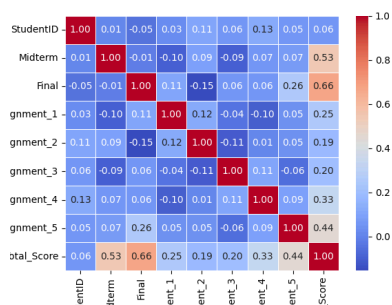
4. Scatter plot of Midterm vs Final, colored by the department. Sample:



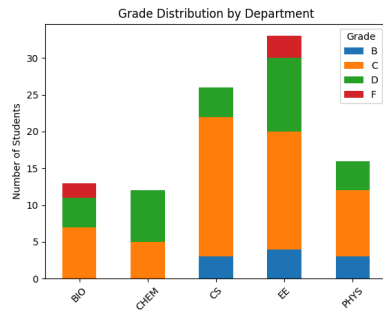
5. Pie chart showing the distribution of students in different departments. Sample:



6. Heatmap of the correlation matrix for the numeric columns (Midterm, Final, total, and the 5 assignments). Sample:



7. Stacked bar chart showing the grade distribution (A, B, C,...) by department. Sample:



Deliverables

Students should submit:

1. **Complete Python script** (`university_course_performance_dashboard.py`)
2. **Documentation** explaining how to use the system